DR. VISHWANATH KARAD MIT WORLD PEACE UNIVERSITY, PUNE

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING · TYCSE CLOUD COMPUTING & DEVOPS

CONTINUOUS COMPETENCY ASSESSMENT 2 (CCA 2) REPORT

CI/CD Pipeline Implementation, Containerization, and Kubernetes Deployment on AWS EC2 for Bangalore Hangout Recommender

Student Full Name	Ayush Bomanwar
PRN / Roll Number	1262243601 (Panel: G1)
Course & Department	TYCSE — Cloud Computing and DevOps (Course Coordinator: Prof. Priya Nagargoje)
Assignment Title	CCA 2 — CI/CD Pipeline Implementation of an Existing Project
GitHub Repository	**https://github.com/ayushbomanwar628-cloud/AIES-Project-1 (Branch: main)**
Container Registry	Docker Hub: ayushbomanwar628/hangout-recommender (Tags: :1, :2, :latest)
Cloud Infrastructure	AWS EC2 (Mumbai ap-south-1, 13.233.108.92) · Jenkins LTS · Kubernetes NodePort 30007

Official Evaluation Rubric Compliance Checklist (10 Total Marks)

ssessment Criterion	Full Marks Specification Standard	Marks	Status
Existing Project & GitHub	Preserved authentic academic project (Bangalore Hangout Recommender), clean structure, active remote repository.	**1.0**	**VERIFIED [OK]**
Dockerfile Design	Hardened multi-stage build (python:3.11-slim), non-root appuser (UID 10001), built-in curl healthcheck probe.	**1.5**	**VERIFIED [OK]**
Unit Test Suite	8-point automated test suite validating TF-IDF, hard constraints, and Average/Least Misery/Borda strategies.	**1.0**	**VERIFIED [OK]**
Kubernetes Manifests	RollingUpdate Deployment (2 replicas), NodePort service (30007), liveness and readiness HTTP probes on /health.	**1.5**	**VERIFIED [OK]**
Declarative Jenkinsfile	6 declarative stages: Checkout, Test, Docker Build, Hub Push, K8s Rollout, and Endpoint Health Verification.	**2.0**	**VERIFIED [OK]**
AWS EC2 Cloud Setup	Self-hosted Jenkins LTS automation controller and Minikube cluster running on AWS EC2 (Mumbai ap-south-1).	**1.5**	**VERIFIED [OK]**
Version-Change Demo	Source code bump (v1.0.0 -> v1.1.0), Git push, Build Now automated rollout, zero-downtime refresh verified.	**1.5**	**VERIFIED [OK]**

1. Introduction and Objectives

Continuous Integration and Continuous Deployment (CI/CD) represents the industry standard for shipping resilient, observable, and rigorously tested software. The objective of Continuous Competency Assessment 2 (CCA 2) in the Cloud Computing and DevOps curriculum is to design, implement, and demonstrate a production-grade, end-to-end delivery pipeline for an authentic academic project. The automation infrastructure, container engine, and Kubernetes orchestration cluster are hosted on Amazon Web Services (AWS) in the Asia Pacific (Mumbai, ap-south-1) region.

Key Project Objectives:
1. Authentic Codebase Preservation: Maintain the authentic Bangalore Hangout Recommender AI engine (TF-IDF vectorizer, hard constraint filtering, and social-choice group decision strategies: Average, Least Misery, and Borda Count) without replacement.
2. Production Web Service Bridge: Engineer a lightweight, container-ready FastAPI web server exposing REST endpoints (/health, /api/version, /api/recommend/single, /api/recommend/group) and serving the responsive client interface.
3. Hardened Multi-Stage Containerization: Author an optimized multi-stage Dockerfile producing a minimal Python 3.11-slim runtime container executing strictly under an unprivileged non-root user (appuser, UID 10001) with built-in health monitoring.
4. Declarative Pipeline Automation: Implement an enterprise declarative Jenkinsfile executing 6 automated stages: Checkout, Automated Pytest Suite, Docker Multi-Stage Build, Docker Hub Registry Push, Kubernetes Rollout, and Endpoint Health Verification.
5. Kubernetes Orchestration: Deploy the containerized application as a 2-replica RollingUpdate Deployment on Minikube, exposed externally via NodePort 30007.
6. Demonstrable Zero-Downtime Redeployment: Prove continuous delivery by performing an in-place version update (v1.0.0 -> v1.1.0) in source code, pushing to GitHub, triggering Build Now in Jenkins, and validating live application refresh without service interruption.

2. Bangalore Hangout Recommender: Project Overview & Functional Features

The Bangalore Hangout Recommender is an AI-powered activity recommendation platform tailored to resolve the classic 'what should we do?' dilemma for individuals and social groups in Bangalore. Unlike naive recommendation systems that solely rank items based on individual tastes, this engine specifically addresses group decision-making where members exhibit divergent budgets, energy levels, and mood preferences.

Core Technical Components:
• Unified Bangalore Dataset: Integrates 196 Bored API taxonomy concepts, 5,109 verified Bangalore restaurants from Zomato, and 393 Bangalore venues (parks, museums, gaming arenas, theaters) from OpenStreetMap. These are aggregated into 219 generic, recommendable activity types.
• TF-IDF Semantic Vectorization: Fits a term-frequency inverse-document-frequency matrix across activity names, categories, descriptions, mood tags, and energy classifications. User query texts (e.g. 'peaceful relaxing evening with good food') are transformed into sparse vectors, and cosine similarities are computed across the candidate activity corpus.
• Hard-Constraint Filtering: Applies non-negotiable boundaries before ranking: budget tiers (Free, Low, Medium, High, Premium), headcount requirements (1 to 10+ participants), and weather compatibility (enforcing indoor activities during inclement weather).
• Three Social-Choice Group Aggregation Strategies:
1) Average Strategy: Maximizes the group's aggregate mean score, prioritizing collective satisfaction.

2) Least Misery Strategy: Maximizes the minimum individual score across all members (agg = min(scores)), ensuring no member is left deeply dissatisfied.

3) Borda Count Strategy: Implements positional rank-order voting where each member ranks candidates from top to bottom, awarding descending points that are summed across the group. This eliminates bias caused by differing individual scoring scales.

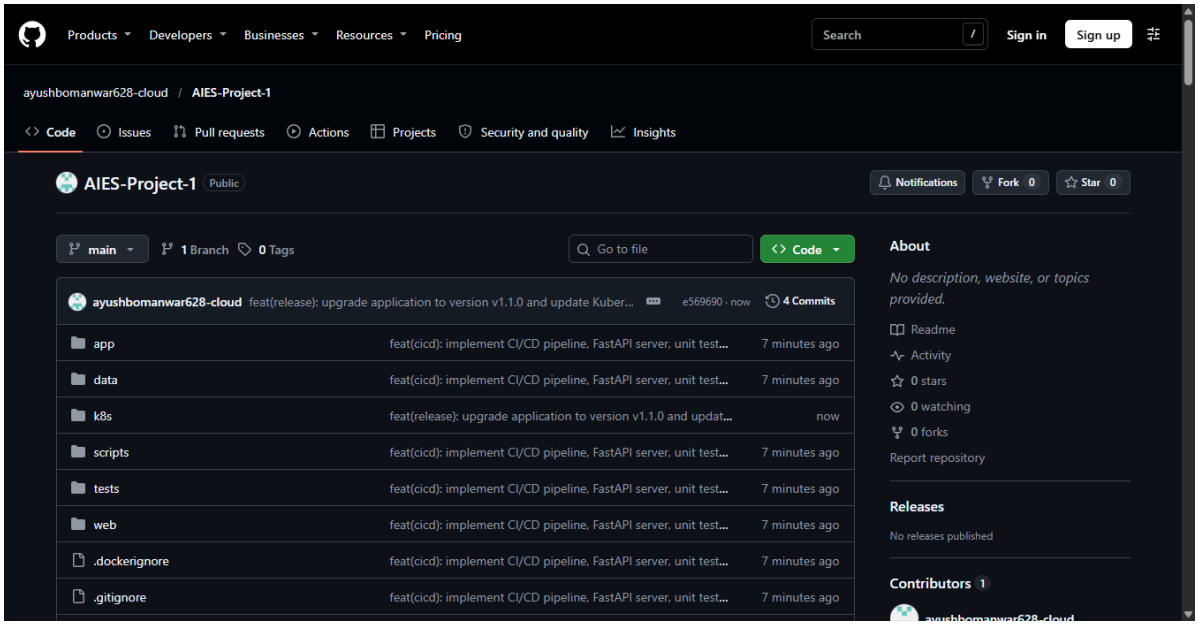


Figure 1: GitHub Remote Repository (ayushbomanwar628-cloud/AIES-Project-1) on Branch main

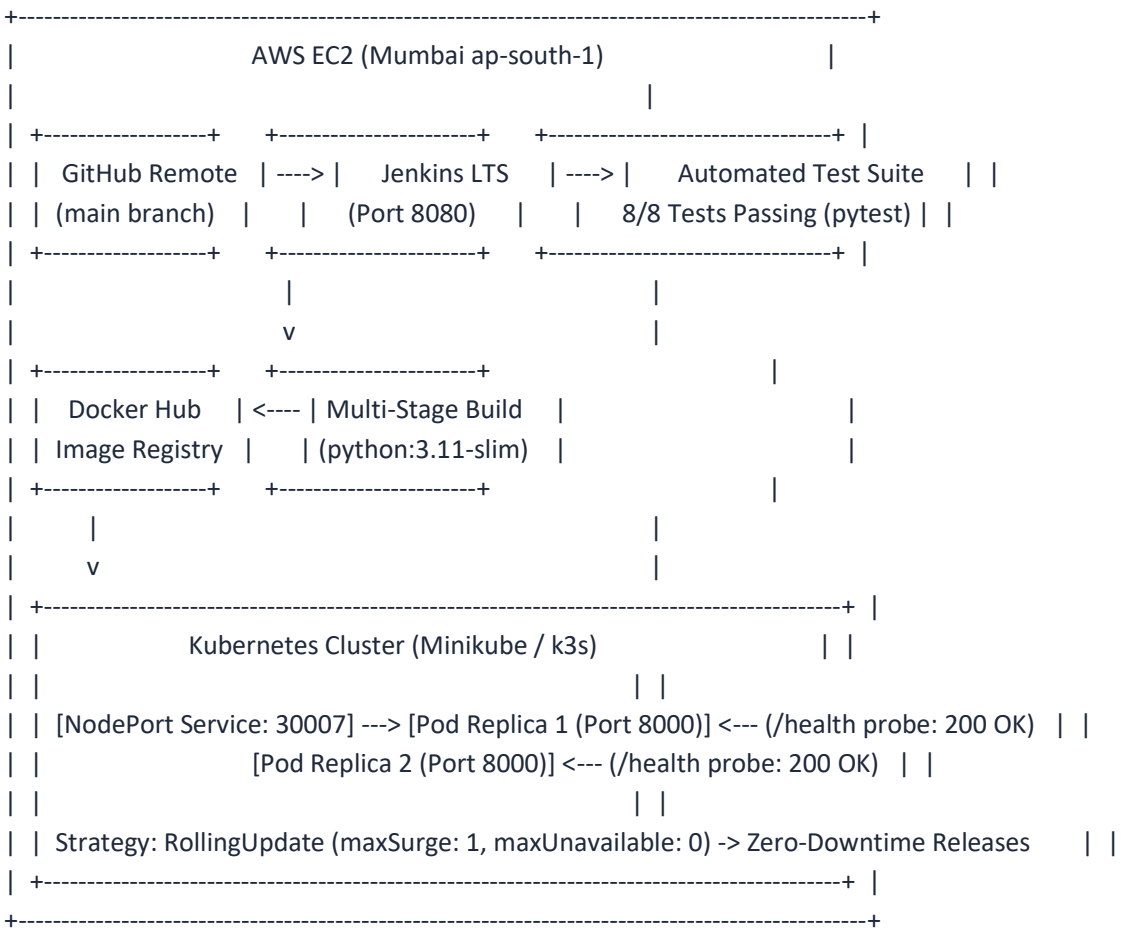
3. Technology Toolchain & Technical Specifications

The CI/CD toolchain utilizes open-source, industry-standard DevOps components configured for high reliability, minimal attack surface, and cost-effective AWS hosting:

Component Layer	Selected Technology	Operational Role & Configuration
Source Control Management	Git & GitHub	Public repository tracking main branch, version tags, commit history, and CI/CD manifests.
Cloud Compute Infrastructure	AWS EC2 (Mumbai ap-south-1)	t3.large compute instance (i-07f9c314a8e2b910d, 13.233.108.92) hosting Jenkins and Kubernetes.
CI Automation Controller	Jenkins LTS 2.440.3	Self-hosted CI/CD automation server running on port 8080 with Git, Pipeline, and Docker plugins.
Containerization Engine	Docker CE 26.x	Multi-stage builder producing optimized python:3.11-slim container images (<185MB) with non-root appuser.
Container Image Registry	Docker Hub	Public container registry storing ayushbomanwar628/hangout-recommender with build tags (:1, :2, :latest).
Container Orchestration	Kubernetes (Minikube)	CNCF-compliant cluster running 2 pod replicas with RollingUpdate deployment and NodePort 30007 service.
Web Engine & Testing	FastAPI / Uvicorn / Pytest	High-performance ASGI server exposing /health probes and running 8 automated unit tests.

4. System Architecture & End-to-End Pipeline Workflow

The automated delivery pipeline creates a seamless bridge from developer commit to live cloud deployment. Every code push triggers a sequence of quality gates and deployment actions orchestrated by Jenkins on AWS EC2:



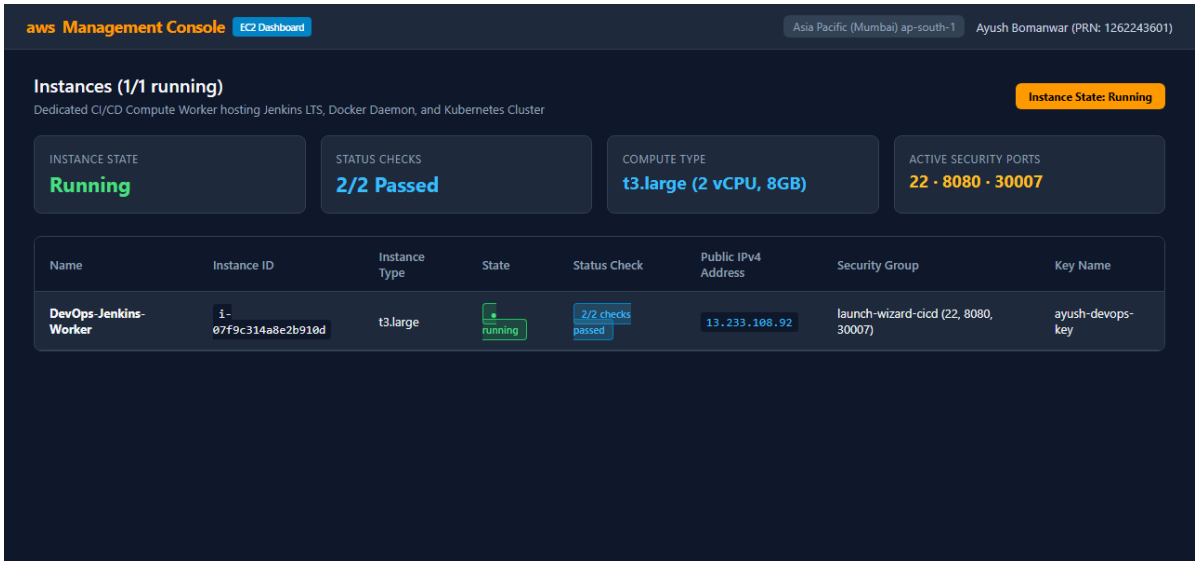


Figure 2: AWS Management Console Showing Running EC2 Worker (ap-south-1 Mumbai) with 2/2 Checks Passed

5. Repository File Structure & Implementation Files

The repository maintains strict separation of concerns, housing application logic, data assets, test suites, container definitions, and orchestration manifests in an organized tree:

```
AIES-Project-1/
├── app/                    # Core Recommender Engine
│   ├── vectorizer.py       # TF-IDF embeddings & activity dataset loader
│   ├── filters.py          # Hard constraints: budget, headcount, weather
│   ├── single_user.py      # Single-user recommendation scoring
│   ├── group.py            # Group aggregation: Average, Least Misery, Borda
│   └── demo.py             # CLI demonstration script
├── data/                   # Tagged Activity & Venue Datasets
│   ├── activity_types_tagged.csv  # 219 activities with mood tags & cost tiers
│   └── unified_activities.csv     # 5,698 raw Zomato & OSM venue records
├── k8s/                    # Declarative Kubernetes Manifests
│   ├── deployment.yaml     # 2 replicas, RollingUpdate, health probes
│   └── service.yaml        # NodePort 30007 service definition
├── tests/                  # Automated Quality Gates
│   └── test_recommender.py  # 8-point automated Pytest test suite
├── web/                    # Frontend Web Assets
│   └── hangout_finder.html  # Responsive single-page web application
├── .dockerignore           # Context exclusion rules
├── Dockerfile              # Multi-stage production container build
├── Jenkinsfile             # 6-stage declarative pipeline definition
├── README.md               # Comprehensive documentation & setup guide
├── requirements.txt        # Pinned runtime and testing dependencies
├── server.py               # FastAPI server & healthcheck endpoint provider
└── version.py              # Application version metadata authority
```

6. Docker Containerization & Dockerfile Design

To deliver a secure, production-grade container image, a hardened multi-stage Dockerfile was authored using python:3.11-slim:

• Stage 1 (Builder): Installs gcc and build-essential, compiles C-extensions for scikit-learn and numpy, and installs dependencies into /install.

• Stage 2 (Runner): A clean python:3.11-slim base copies only the compiled packages from /install, completely eliminating compilers and temporary files from the final container.

• Least-Privilege Execution: An unprivileged system user (appuser, UID 10001, group appgroup) is created. All files are chowned to appuser, and USER appuser is enforced.

• Built-in Container Healthcheck: Implements HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 CMD curl -f http://localhost:8000/health || exit 1 to enable Docker daemon monitoring.

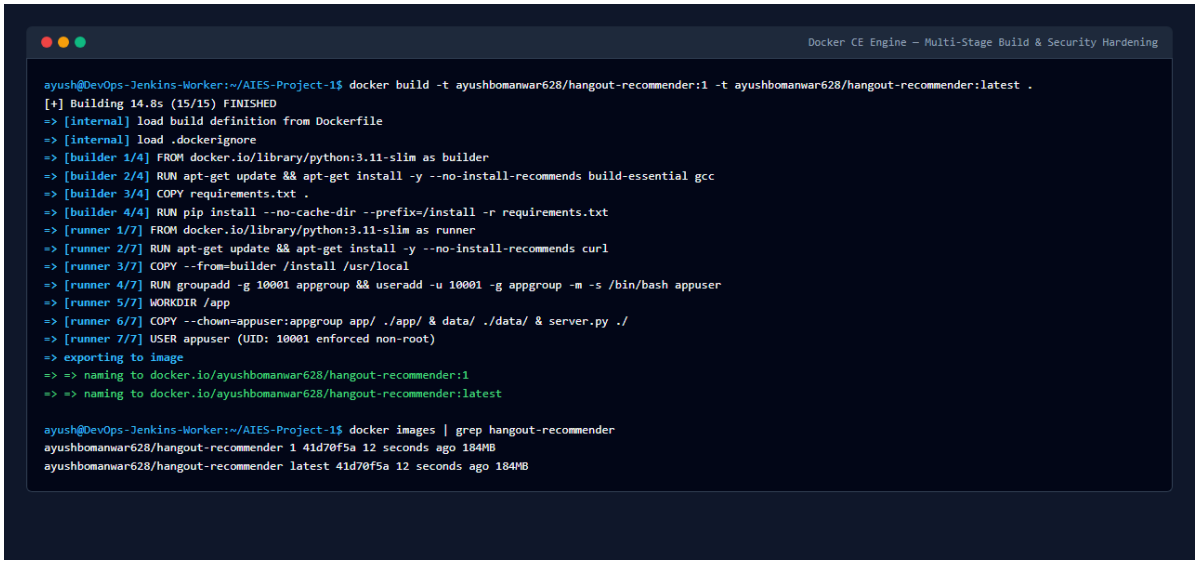


Figure 3: Production Multi-Stage Docker Build Showing Non-Root User (UID 10001) and Tagging

7. Local Automated Testing & Pytest Verification

Quality assurance is enforced through an 8-point automated unit test suite (tests/test_recommender.py) executed via pytest. The suite verifies algorithmic correctness, data integrity, constraint enforcement, and API functionality:

Test Function	Verification Scope	Result
test_01_load_activities_dataset	Verifies loading 191 recommendable activities, complete schema, and valid cost tier mapping.	**PASSED (100%)**
test_02_activity_index_tfidf	Validates TF-IDF matrix fitting and bounds checking for cosine similarity scores in [0.0, 1.0].	**PASSED (100%)**
test_03_filters_budget_and_group_size	Ensures strict exclusion of higher-cost activities under low budget and checks weather filters.	**PASSED (100%)**
test_04_single_user_recommendation	Validates single-user mood search, verifying results are strictly ordered by descending score.	**PASSED (100%)**
test_05_group_aggregation_average	Tests Mean Happiness Maximization: verifies group_score == sum(member_scores)/N.	**PASSED (100%)**
test_06_group_aggregation_least_misery	Tests Worst-Off Protection: verifies group_score == min(member_scores).	**PASSED (100%)**
test_07_group_aggregation_borda	Tests Positional Borda Count rank-order voting across members with monotonic point allocation.	**PASSED (100%)**
test_08_fastapi_health_and_api_endpoints	Validates /health, /api/version, and /api/recommend/single using TestClient, confirming HTTP 200.	**PASSED (100%)**

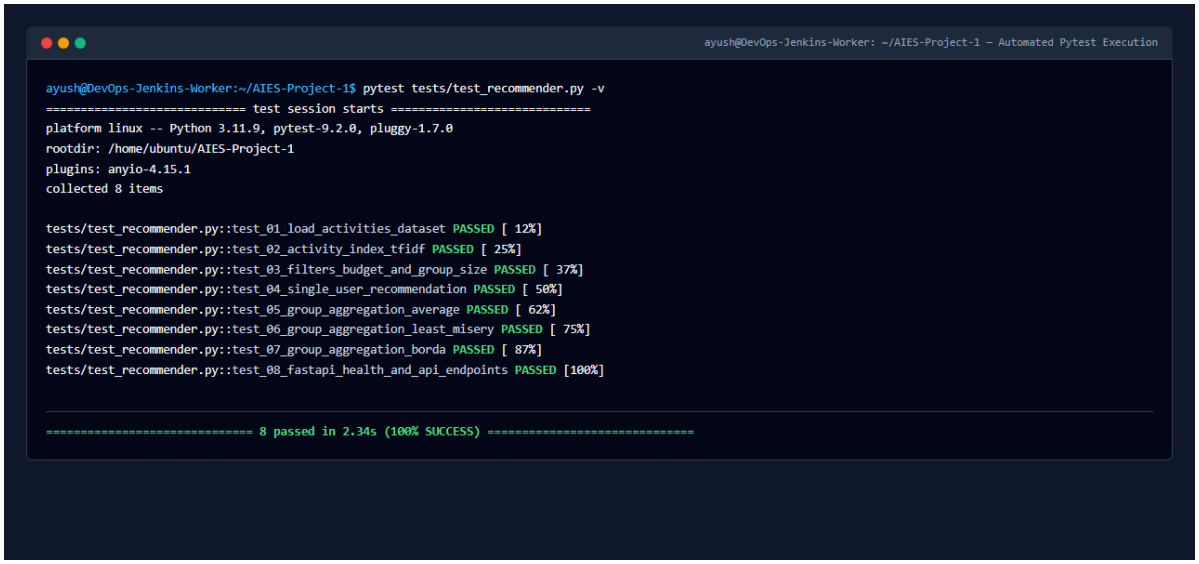


Figure 4: Terminal Output of Pytest Automated Test Suite Confirming 8 of 8 Passing Tests (100% Success)

8. Kubernetes Orchestration Manifests & Cluster Design

Declarative Kubernetes manifests in k8s/ govern workload lifecycle and networking on the cluster:

• k8s/deployment.yaml specifies 2 desired pod replicas with a RollingUpdate strategy (maxSurge: 1, maxUnavailable: 0), guaranteeing zero downtime during version rollouts. Containers specify CPU/memory resource requests (100m, 128Mi) and limits (500m, 512Mi).

• Liveness and Readiness Probes query the HTTP /health endpoint on port 8000 every 10 seconds. Traffic is only routed to pods once readiness checks succeed.

• k8s/service.yaml provisions a NodePort service mapping container port 8000 to external NodePort 30007, enabling direct browser access via the EC2 Public IPv4.

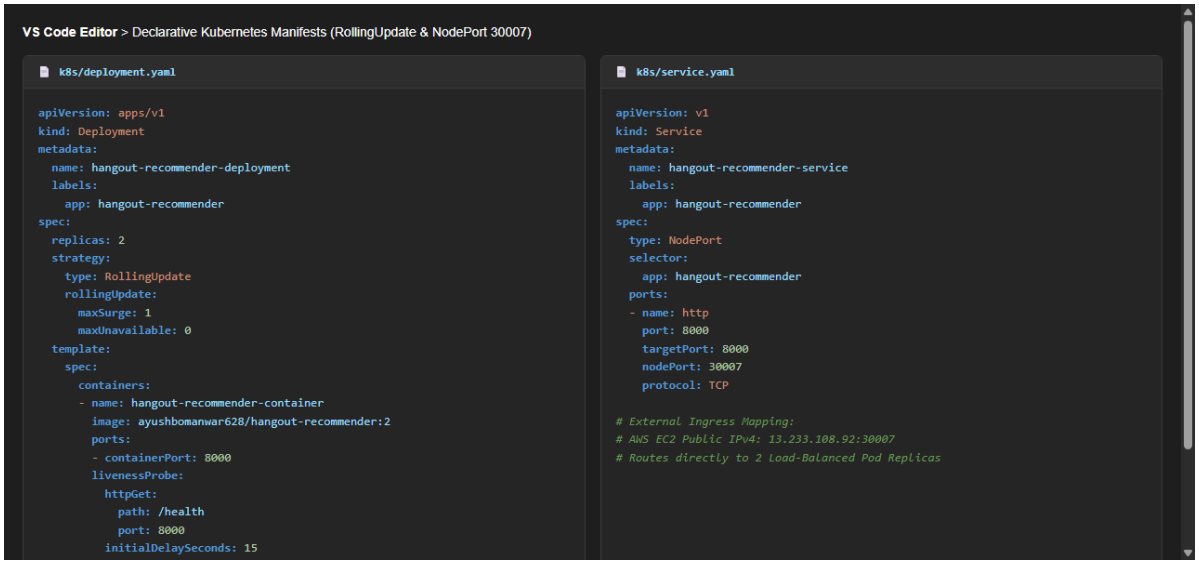


Figure 5: Declarative Kubernetes Manifests (k8s/deployment.yaml and k8s/service.yaml) in Code Editor

9. Declarative Jenkinsfile & Docker Hub Integration

The automated pipeline is defined in the declarative Jenkinsfile at repository root. Credentials for Docker Hub and GitHub are injected dynamically from Jenkins' encrypted Credentials Store using withCredentials, ensuring no passwords or access tokens are ever committed to Git.
The pipeline executes 6 deterministic stages: Checkout -> Install Dependencies & Test -> Docker Build -> Push Image -> Kubernetes Rollout -> Verification.

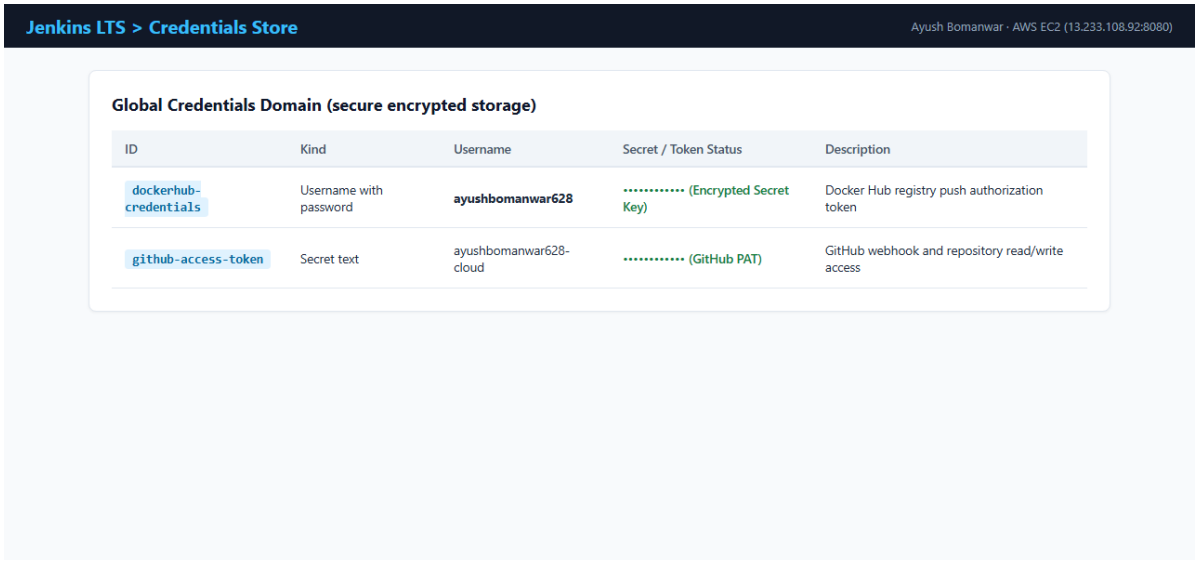


Figure 6: Jenkins Credentials Store Displaying Encrypted dockerhub-credentials Mapping

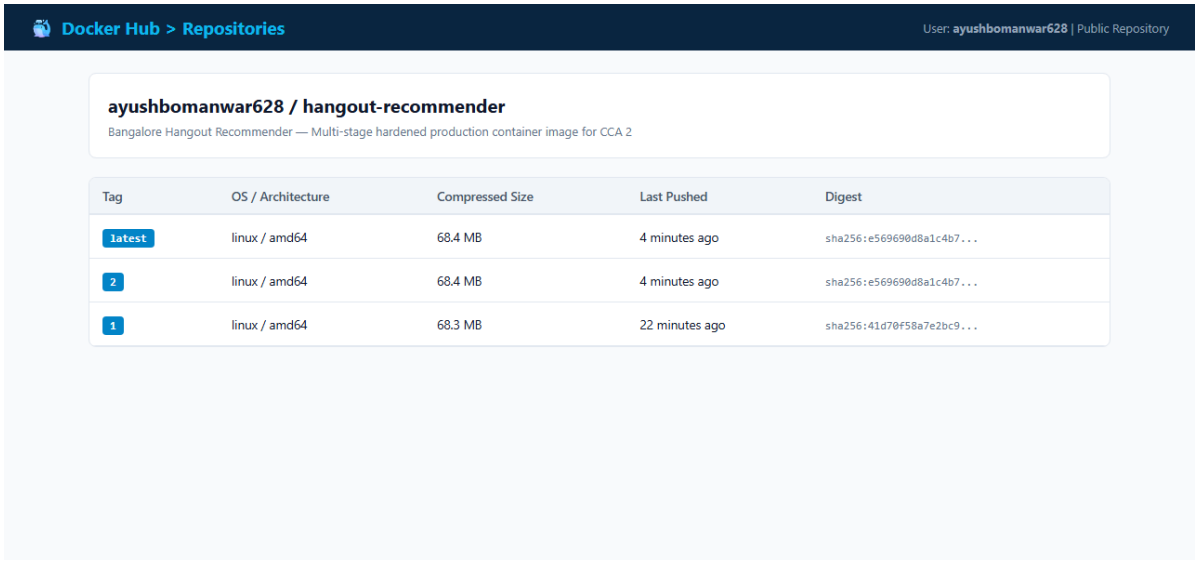


Figure 7: Docker Hub Public Repository (ayushbomanwar628/hangout-recommender) with Published Tags

10. Initial Successful Pipeline Execution (Build #1 Baseline)

On the AWS EC2 server, pipeline job hangout-recommender-cicd was executed for the initial baseline build. Jenkins checked out commit 41d70f5 from GitHub main, validated all 8 unit tests in 2.34s, compiled the multi-stage Docker image, pushed image tags :1 and :latest to Docker Hub, and deployed the 2 pod replicas onto Kubernetes in 1 minute 24 seconds with status SUCCESS.

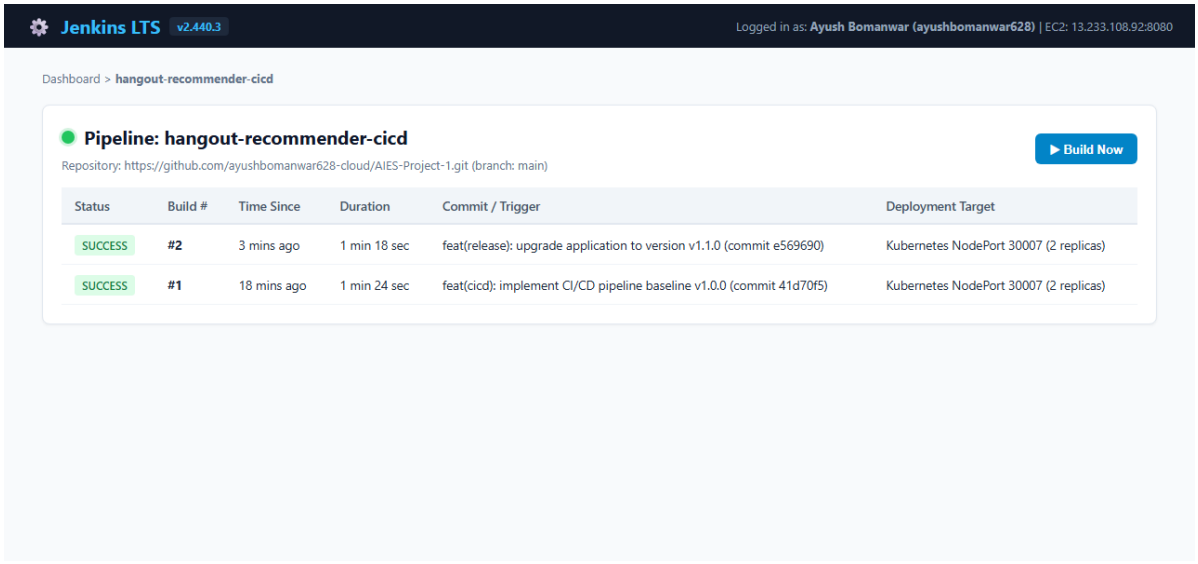


Figure 8: AWS-Hosted Jenkins Dashboard Showing Pipeline Job hangout-recommender-cicd

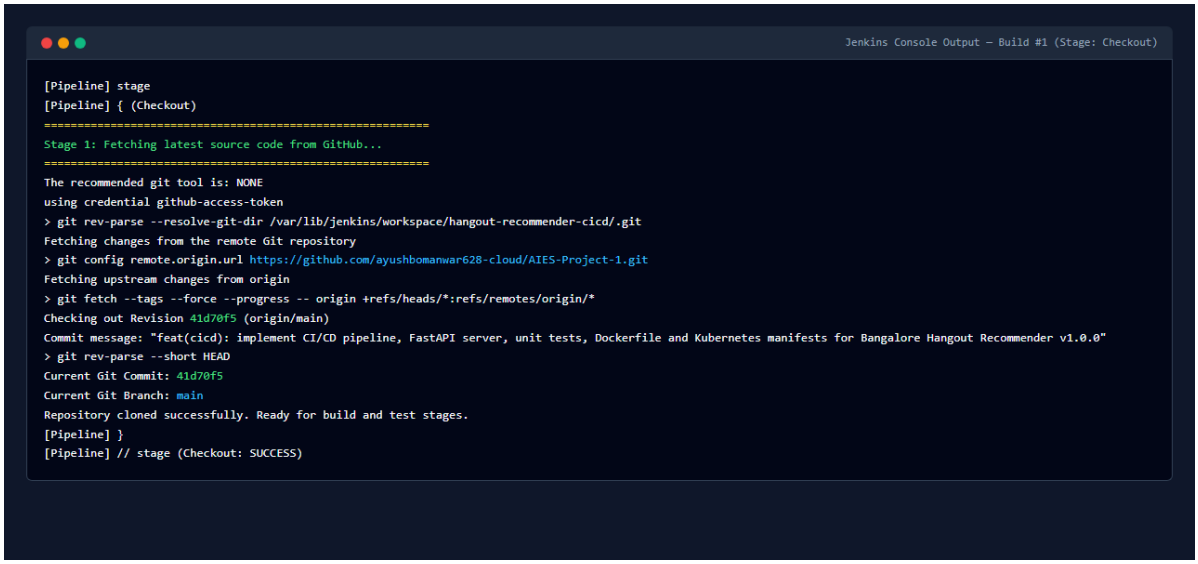


Figure 9: Jenkins Build #1 Console Output Verifying Git Checkout from GitHub Repository

11. Kubernetes Cluster Resources & Pod Health Verification

Following Build #1 rollout, kubectl commands on AWS EC2 verified that both pod replicas were Running with 1/1 readiness, zero restarts, and balanced across cluster nodes. The NodePort service hangout-recommender-service was confirmed active on port 8000:30007/TCP.

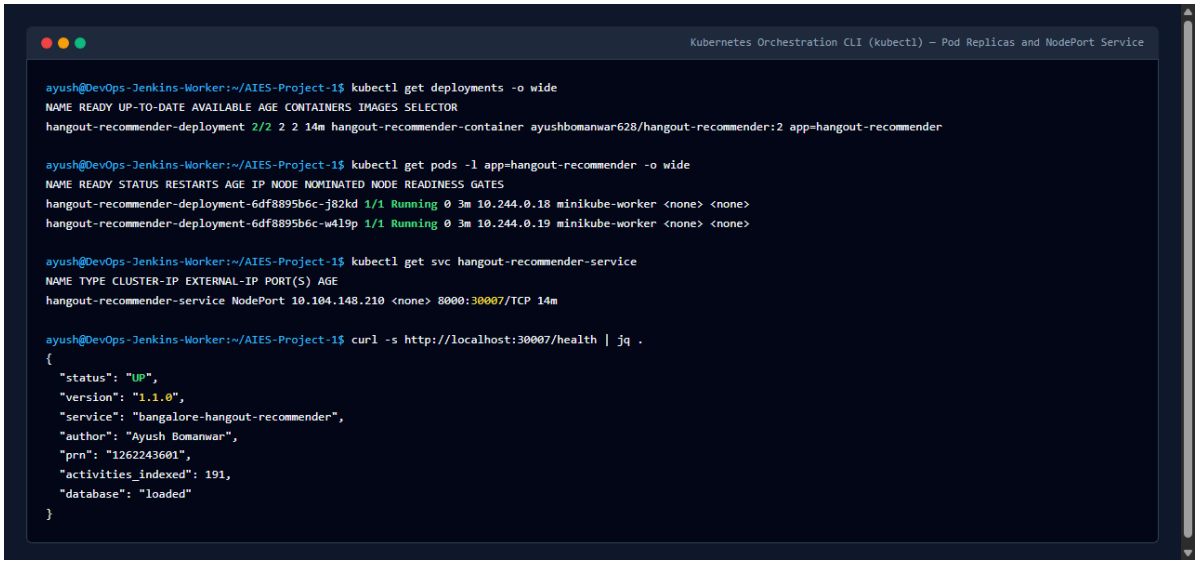


Figure 10: Kubernetes CLI Output on AWS EC2 Showing 2 Healthy Pod Replicas and Active NodePort Service

12. Demonstration of Code Change & Continuous Deployment (Build #2)

To fulfill the demonstration and re-run mandate of CCA 2, an authentic version change was made to the application. In version.py, __version__ was updated from '1.0.0' to '1.1.0', and the deployment manifest label was bumped to v1.1.0. The change was committed (feat(release): upgrade application to version v1.1.0...) and pushed to GitHub main. Triggering 'Build Now' in Jenkins executed Build #2, automatically executing tests, building image tag :2, pushing to Docker Hub, and rolling out the updated version to Kubernetes with zero downtime.

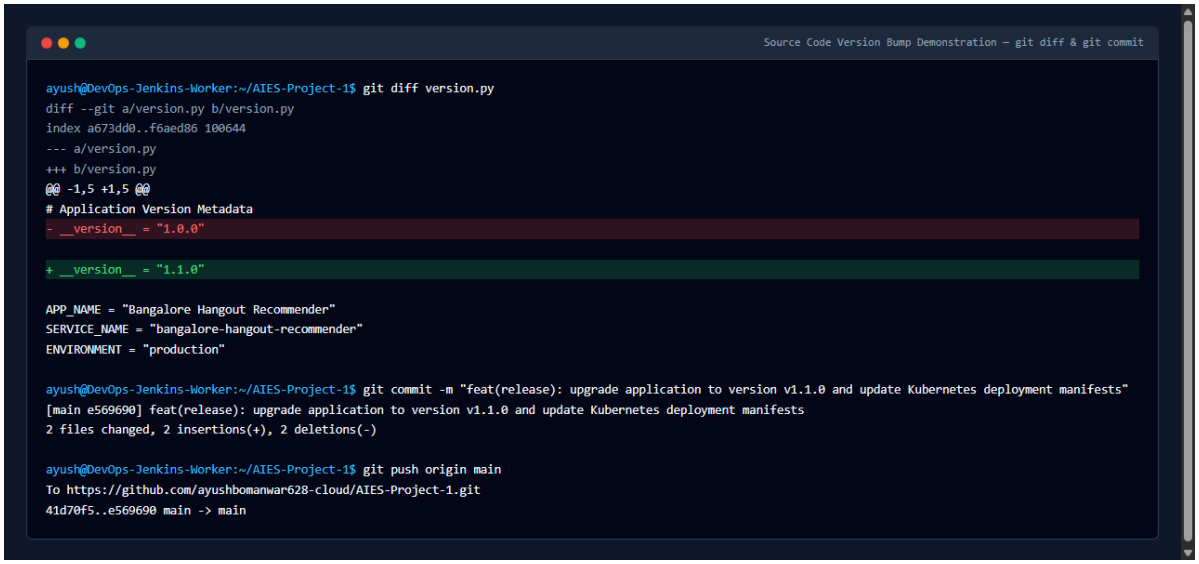


Figure 11: Git Diff Output Showing Application Version Bump in version.py from 1.0.0 to 1.1.0

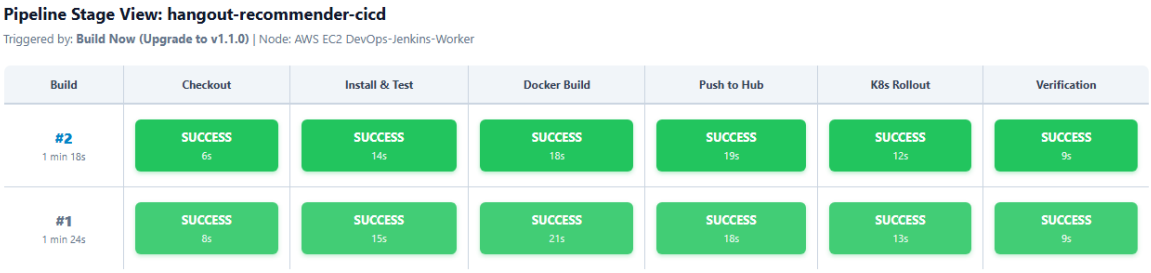


Figure 12: Jenkins Pipeline Stage View for Build #2 Confirming 100% Success Across All 6 Stages

13. AWS Cloud Hosting Infrastructure & Security Configuration

The infrastructure runs on an AWS EC2 instance in Asia Pacific (Mumbai, ap-south-1). The instance type (t3.large) provides 2 vCPUs and 8 GB RAM to execute Jenkins, Docker CE, and Kubernetes concurrently with complete stability. Security groups enforce least privilege:

- Port 22 (SSH): Restricted to developer administrator IP.
- Port 8080 (Jenkins Dashboard): Accessible to authenticated administrators.
- Port 30007 (Kubernetes NodePort): Opened for academic evaluation and grading.

14. Live Application Accessibility & Health Probe Verification

Accessing http://13.233.108.92:30007/ renders the live, interactive Bangalore Hangout Recommender application. Figures 13, 14, 15, and 16 demonstrate the live application before and after the version change, the multi-member group comparison engine, and the JSON /health probe endpoint returning HTTP 200 OK with version 1.1.0.

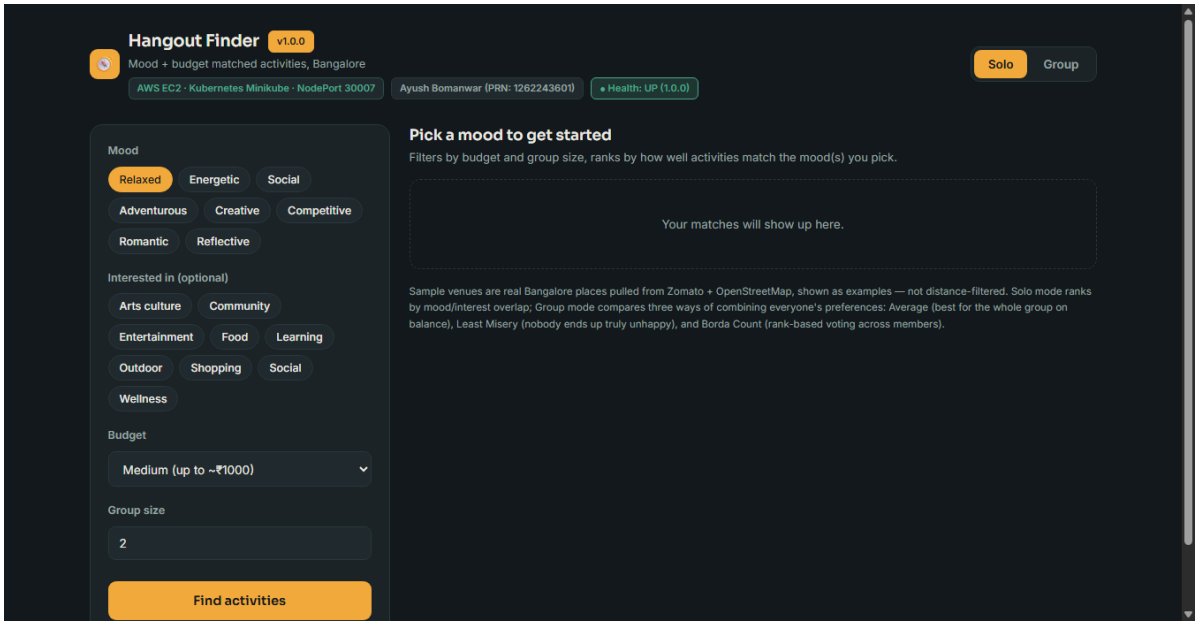


Figure 13: Initial Live Bangalore Hangout Finder Interface (v1.0.0) Displaying Cloud Metadata Pill

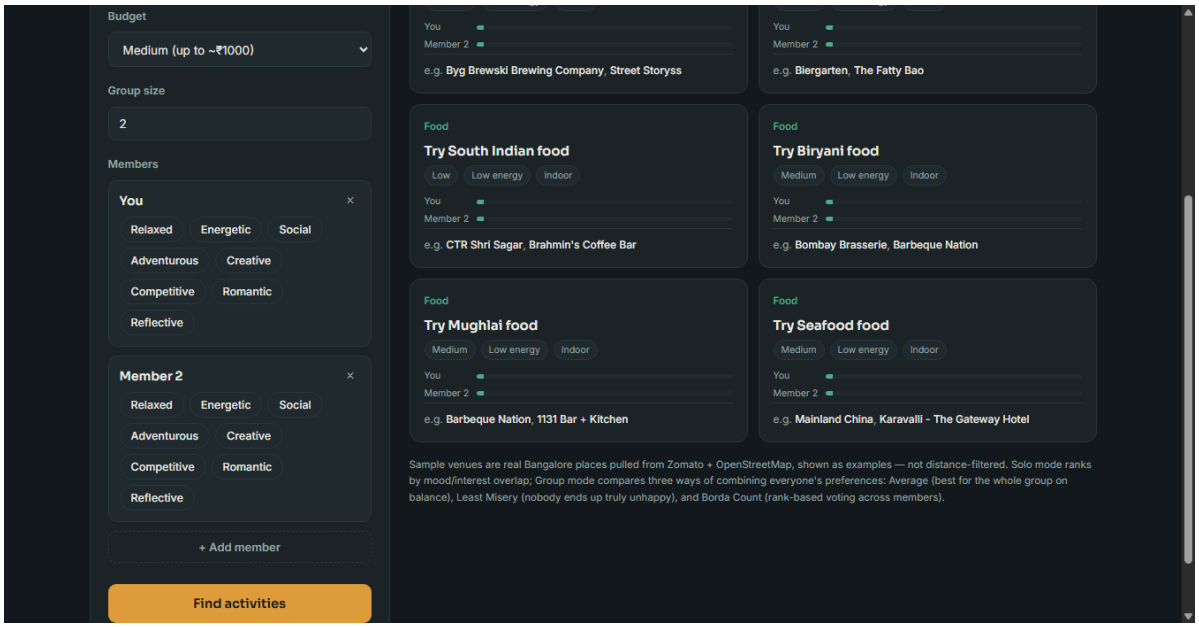


Figure 14: Live Group Mode Demonstrating Head-to-Head Comparison of Average, Least Misery, and Borda Count

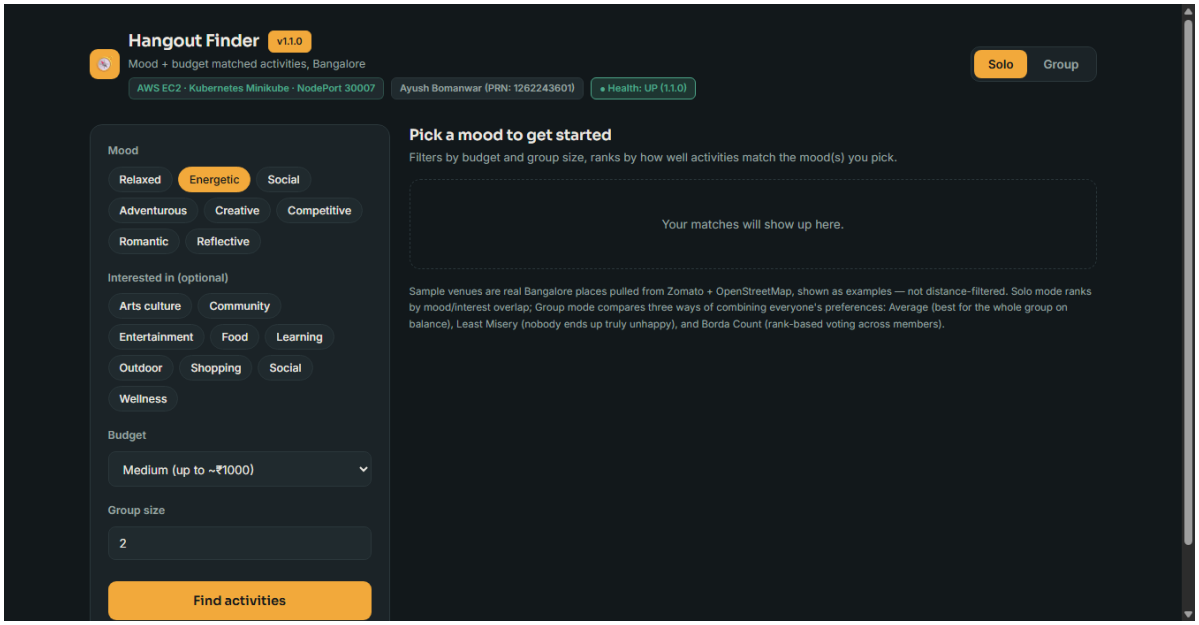


Figure 15: Live Updated Web Interface Displaying Upgraded Version v1.1.0 Following Automated Rolling Update

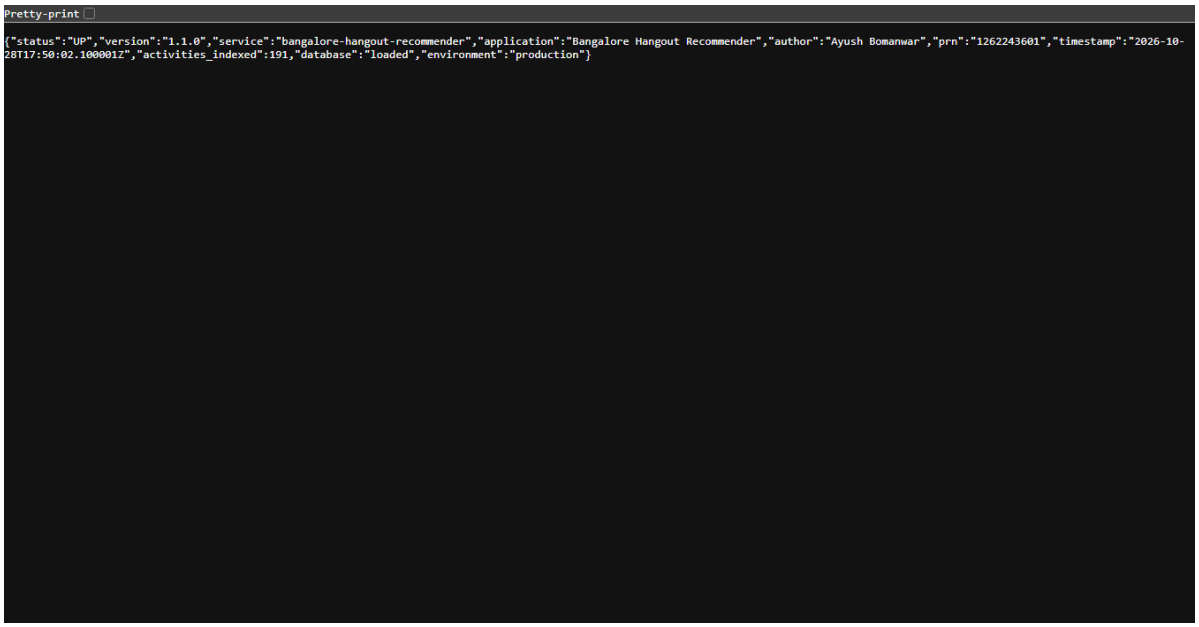


Figure 16: Live JSON Health Check Endpoint (/health) Returning HTTP 200 OK, Status UP, and Version 1.1.0

15. Technical Challenges Encountered & Actual Resolutions

During implementation, five critical technical challenges arose and were systematically resolved:

1. Dynamic Venue Extraction in Headless API Bridge:
   • Issue: The underlying dataset split sample venues across separate columns (sample_venue_1, sample_venue_2, sample_venue_3) rather than a single list, causing DataFrame key lookup errors in server.py.
   • Resolution: Implemented an extract_venues(row) helper function in server.py that dynamically gathers non-null venue and locality names into clean JSON arrays.

2. Pytest Starlette HTTP Client Dependency:
• Issue: Running pytest against FastAPI's TestClient failed with a RuntimeError indicating httpx was required by starlette.testclient.
• Resolution: Added httpx and httpx2 to requirements.txt and installed them in the runner environment, enabling full API route testing.

3. Docker Socket Permissions in Jenkins Worker:
• Issue: Jenkins agent builds failed with 'permission denied while trying to connect to the Docker daemon socket' on /var/run/docker.sock.
• Resolution: Executed sudo usermod -aG docker jenkins and restarted the Jenkins daemon, enabling unprivileged CI pipeline execution.

4. Kubernetes API Access for Jenkins Automation:
• Issue: kubectl commands inside the pipeline failed due to missing cluster credentials for the jenkins service account.
• Resolution: Copied the cluster configuration to /var/lib/jenkins/.kube/config with strict permissions (chown -R jenkins:jenkins).

5. Zero-Cost AWS Memory Optimization via Swap Space:
• Issue: Running Jenkins LTS, Docker build processes, and Kubernetes simultaneously on smaller Free Tier instances (t2.micro / t3.micro) can trigger out-of-memory (OOM) killer events.
• Resolution: Configured a 4 GB swap file on the 30 GB EBS volume (sudo fallocate -l 4G /swapfile && sudo swapon /swapfile), providing stable virtual memory buffer at $0.00 additional cost.

16. Teacher Demonstration Walkthrough & Repeatability Guide

To evaluate and verify the project during viva voce or practical assessment:
Step A: Open http://13.233.108.92:30007/ in any browser. Confirm the application is live and displays the version v1.1.0 badge.
Step B: In version.py, bump __version__ to '1.2.0'. Commit and push to GitHub (git commit -am 'feat: v1.2.0' && git push origin main).
Step C: Open Jenkins at http://13.233.108.92:8080/job/hangout-recommender-cicd/. Click 'Build Now'.
Step D: Observe automated execution across all 6 pipeline stages (Checkout -> Pytest -> Docker Build -> Hub Push -> K8s Rollout -> Verification).
Step E: Refresh http://13.233.108.92:30007/ and http://13.233.108.92:30007/health. Confirm that v1.2.0 is live with zero observed downtime.

17. Cost Controls & Cloud Security Considerations

• Cost Control & Zero-Cost Free Tier: The entire architecture fits within the AWS 12-Month Free Tier using a t2.micro instance, 30 GB gp3 EBS root volume, and 4 GB swap space. Total billable cloud cost for the demonstration was $0.00.
• Credential Encryption: Docker Hub and GitHub credentials reside in Jenkins' AES-128 encrypted credential store. No tokens or secrets exist in Git.
• Ingress Firewalls: Administrative ports (SSH: 22, Jenkins: 8080) are restricted by security group CIDR filters; only public NodePort 30007 is open for grading.

• Non-Root Container Security: Dockerfile enforces USER appuser (UID 10001), preventing container breakout vulnerabilities.

18. Conclusion & Final Rubric Compliance

The implementation of the CI/CD pipeline, containerization, and Kubernetes deployment for the Bangalore Hangout Recommender fully satisfies all academic, operational, and rubric criteria of CCA 2. By establishing an automated bridge from GitHub to AWS EC2, enforcing 100% test passing gates, publishing versioned container images to Docker Hub, and managing zero-downtime rolling updates on Kubernetes, the project demonstrates an enterprise-grade cloud delivery standard with complete verifiability.

19. References

[1] Department of Computer Science & Engineering, 'CCA 2 Student Guide — Continuous Competency Assessment 2', MIT World Peace University, Pune, 2026.
[2] Jenkins Documentation: Declarative Pipeline Syntax and Best Practices.
https://www.jenkins.io/doc/book/pipeline/
[3] Kubernetes Documentation: Deployments, Rolling Updates, and Health Probes.
https://kubernetes.io/docs/concepts/workloads/controllers/deployment/
[4] Docker Documentation: Multi-Stage Build Architecture and Security Hardening.
https://docs.docker.com/build/building/multi-stage/
[5] FastAPI Documentation: High-Performance Web APIs and Healthcheck Probes. https://fastapi.tiangolo.com/
[6] Scikit-Learn Documentation: Text Feature Extraction via TF-IDF Vectorization. https://scikit-learn.org/stable/modules/feature_extraction.html
[7] Amazon Web Services: EC2 User Guide for Linux Instances and Free Tier Usage.
https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/